

UrbanoGo

Decisões técnicas, arquitetura e processo, no início da implementação.

Agenda

-
- 01 Contexto e produto
 - 02 Escopo desta entrega
 - 03 Execução em fatias
 - 04 Decisões de arquitetura (ADRs)
 - 05 Automação garante as decisões
 - 06 Repositório reflete as decisões
 - 07 Entrega no prazo e próximos passos
-

01

— Contexto e produto

Recap e SLA

Recap: o problema



Rafael Souza

precisa mandar um presente para o outro lado da cidade e encontrar, em segundos, um motorista bem avaliado e disponível, mesmo em pico de demanda.

- Problema central: matching e precificação precisam reagir em segundos a mudanças de demanda, chuva, hora do rush. Decisão lenta custa cliente.
- Picos de uso muito marcados: horário de rush, noites de fim de semana, dias de chuva.

A solução

UrbanoGo conecta motoristas parceiros a passageiros e a pequenas entregas dentro da cidade, com geolocalização em tempo real: matching, precificação e rastreamento.



SLA do produto



99,9%

Disponibilidade · ~43min de downtime tolerado por mês



<3s

Matching, ponta a ponta, do pedido ao motorista atribuído



<5s

Latência de posição no mapa, percebida pelo passageiro

Matching e precificação são caminho crítico de baixa latência: precisam ser assíncronos, cacheados e escaláveis horizontalmente.

Bom cenário para arquitetura orientada a eventos, processamento geoespacial e escalabilidade de serviços críticos de baixa latência.

02

— Escopo desta entrega

3 features essenciais, 2 de reserva

Essencial · 1º: Autenticação

- **Problema:** sem saber quem é motorista ou passageiro, nada mais no sistema funciona.
- **Critério:** bloqueante, baixo risco técnico, altíssima alavancagem.
- **Utopia:** login social, MFA, verificação de CNH com biometria.
- **MVP (~3 dias):** cadastro/login por e-mail e senha com JWT, dois papéis.

🔍 Essencial · 2º: Matching

- **Problema:** conectar quem pediu a corrida a um motorista próximo e disponível, rápido.
- **Critério:** core da proposta de valor, caminho crítico de latência.
- **Utopia:** matching multi-critério, leilão dinâmico, balanceamento entre regiões.
- **MVP (~3 dias):** oferta sequencial ao mais próximo, com timeout e fallback.

Essencial · 3º: Rastreamento em tempo real

- **Problema:** sem ver o motorista se movendo, o passageiro perde a confiança.
- **Critério:** alto valor percebido, está na meta de SLA, diferencial visível em demo.
- **Utopia:** GPS de alta precisão colado à rua, push proativo, replay do trajeto.
- **MVP (~3 dias):** WebSocket emitindo posição a cada poucos segundos no mapa.

Ficou de fora, e por quê

- **Avaliação com IA**, não bloqueia a corrida; pode nascer com nota neutra por padrão.
- **Precificação dinâmica**, preço fixo por distância já funciona; ajuste por demanda é otimização.

03

— Execução em fatias

Cada feature, entregue dia a dia

Fatias: Autenticação

Entregue em 3 dias, uma atividade por dia, sempre algo utilizável pelo usuário.

Dia 1	Criar usuário, login e logout: cadastro e autenticação ponta a ponta, com JWT.
-------	--

Dia 2	Definir perfil de usuário: passageiro e motorista com perfis distintos.
-------	---

Dia 3	Reset de senha por e-mail: recuperação de senha integrada.
-------	--

Fatias: Matching

Entregue em 3 dias, uma atividade por dia, sempre algo utilizável pelo usuário.

Dia 4	Solicitar e aceitar corrida/entrega: fluxo básico funcionando.
-------	--

Dia 5	Ranking por proximidade: motorista mais próximo priorizado na oferta.
-------	---

Dia 6	Avaliação mútua: nota pós-corrida por média simples, sem IA.
-------	--

Fatias: Rastreamento em tempo real

Entregue em 3 dias, uma atividade por dia, sempre algo utilizável pelo usuário.

Dia 7	Mapa + posição atual: mapa exibindo a posição do motorista.
-------	---

Dia 8	WebSocket: posição atualizada em tempo real via socket.
-------	---

Dia 9	Suavização do movimento: posição prevista entre atualizações.
-------	---

04

— Decisões de arquitetura

3 ADRs, uma por fatia

ADR 0001: JWT sem estado

Fatia: Autenticação, dia 1

🕒 CONTEXTO

Fatia bloqueante para todo o resto do projeto; decisão precisa ser simples, de baixo risco, sem depender de infraestrutura que ainda não existe.

✅ DECISÃO

Autenticação stateless via JWT: token assinado carrega o id do usuário, validado a cada requisição sem consulta a tabela de sessões. Logout tratado no cliente.

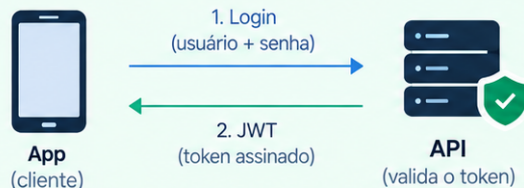
→ CONSEQUÊNCIAS

Escala horizontalmente sem estado no servidor. Logout não revoga o token de fato; revogação exigirá blocklist ou refresh token no futuro.

aceito

ADR 0001: alternativas consideradas

✓ Solução escolhida JWT (stateless)



- ✓ Token assinado carrega o id do usuário.
- ✓ Validado a cada requisição sem consulta a tabela de sessões.
- ✓ Escala horizontalmente sem estado no servidor.

Alternativa 1

Cookie + Redis

Sessão no servidor

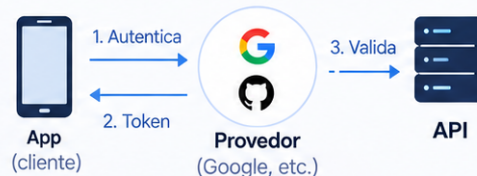


- Exige infraestrutura extra (Redis).
- Maior complexidade de configuração (CORS, expiração, etc.).
- Menos escalável (estado no servidor).

Alternativa 2

OAuth / Login social

Provedor externo (Google, etc.)



- Integração mais complexa.
- Maior risco para entrega de um dia.
- Dependência de terceiros (disponibilidade, políticas, etc.).

ADR 0002: Haversine em SQL

Fatia: Matching, dia 2

🔗 CONTEXTO

Priorizar, entre motoristas disponíveis, o mais próximo do passageiro, entregável em um dia, sem somar infraestrutura nova ao prazo do MVP.

✅ DECISÃO

Distância calculada por haversine direto em SQL no PostgreSQL. Uma bounding box com índice B-tree em (lat, lng) descarta a maior parte das linhas antes do cálculo.

→ CONSEQUÊNCIAS

Não soma dependência nova, simples de testar em um dia. Mais lento que índice espacial nativo conforme o número de motoristas crescer.

aceito

ADR 0002: alternativas consideradas



Solução escolhida

Haversine em SQL (PostgreSQL)

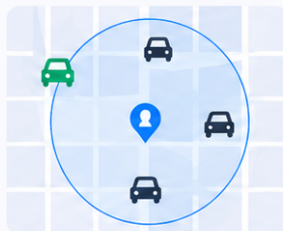


- ✓ Sem dependências adicionais (usa apenas o PostgreSQL).
- ✓ Entregável no prazo do MVP.
- ✓ Atende bem ao volume inicial com boa performance.

Alternativa 1

PostGIS

Extensão geoespacial do PostgreSQL



- Exige instalação e manutenção da extensão PostGIS.
- Mais complexo de configurar.
- Desnecessário para o volume inicial do MVP.

Alternativa 2

Redis GEO

Índices geoespaciais no Redis



- Adiciona nova infraestrutura (Redis).
- Eventual consistência entre banco principal e Redis.
- Mais complexo para o escopo de um dia.

ADR 0003: WebSocket (Socket.IO)

Fatia: Rastreamento em tempo real, dia 2

🕒 CONTEXTO

A fatia anterior só mostrava a posição atual; era preciso escolher o transporte das atualizações entre motorista, servidor e passageiro.

✅ DECISÃO

WebSocket via Socket.IO, com sala isolada por corrida (ride:{rideld}) e autenticação por token no handshake. Servidor retransmite a posição para a sala.

→ CONSEQUÊNCIAS

Atualização quase instantânea, canal único bidirecional. Introduce estado de conexão por sala e exige lidar com reconexão e queda de conexão.

aceito

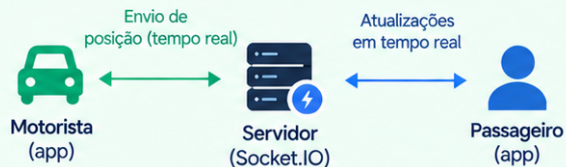
ADR 0003: alternativas consideradas



Solução escolhida

WebSocket (Socket.IO)

Canal bidirecional em tempo real

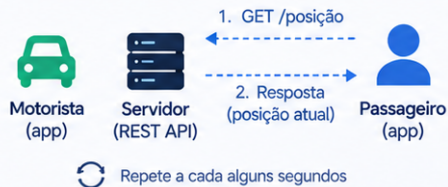


- ✓ Canal único e bidirecional (motorista ↔ passageiro).
- ✓ Baixa latência (atualização em tempo real).
- ✓ Escalável com salas por corrida.

Alternativa 1

Polling HTTP

Requisições periódicas



- ✗ Mais requisições (maior uso de recursos).
- ✗ Maior latência (atualizações em intervalos).
- ✗ Pode gerar sobrecarga em picos.

Alternativa 2

Server-Sent Events (SSE)

Canal unidirecional (servidor → cliente)



- ✗ Apenas um sentido (servidor → cliente).
- ✗ Ainda precisa de outro canal para o motorista enviar a posição.
- ✗ Menos flexível que o WebSocket para interações futuras.

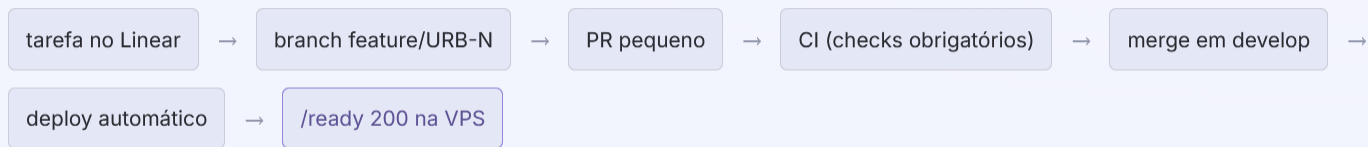
05

— Automação garante as decisões

CI/CD por PR

Nosso fluxo de trabalho

Toda mudança segue este caminho, do planejamento até rodar em produção.



A seguir, o detalhe de cada camada desse fluxo: os checks de CI, a pipeline de segurança e o deploy.

CI por repositório

📁 Backend, 3 workflows

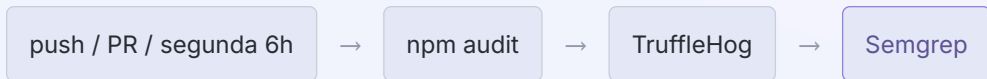
- **ci.yml**, todo push e PR: lint → build → prisma migrate deploy → testes, contra Postgres 17 e Redis 7 reais como services do runner.
- **security.yml**, em push/PR e toda segunda 6h: npm audit, TruffleHog, Semgrep.
- **deploy.yml**, só em push para develop: Ansible provisiona a VPS e sobe a stack.

3 camadas: correção, segurança, entrega

📁 App Flutter, 1 workflow

- **ci.yml**, em push/PR: flutter analyze, lint estático, flutter test e scan de segredo.
Ainda sem SAST, sem scan de dependência, sem pipeline de deploy. Reconhecido como próximo passo, não escondido.

Pipeline de segurança do backend



npm audit

Produção: `--audit-level=high`,
bloqueante.

Geral: `moderate`, `report-only`.

TruffleHog

`Secret scan`, `--results=verified`.

Só reporta segredo confirmado, não
achado ambíguo.

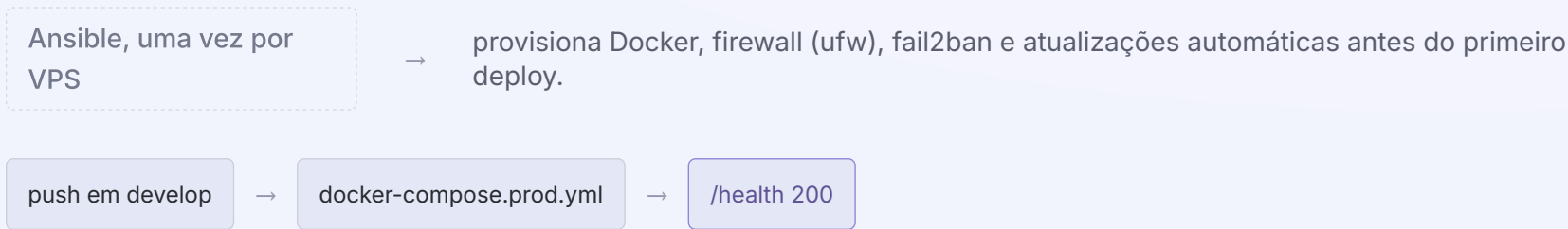
Semgrep (SAST)

Rulesets: `p/typescript`, `p/nodejs`.

`p/javascript`, `p/security-audit`.

app ainda não tem essa camada

Deploy contínuo



- Só em push para develop ou disparo manual, nunca automático em outra branch.
- docker-compose.prod.yml sobe API, Postgres e Redis atrás do Caddy, com TLS automático.

É esse fluxo que a demo ao vivo mostra rodando.

push → produção

06

— Repositório reflete as decisões

Estrutura de pastas

Estrutura de pastas: Backend

Node/Fastify. Módulos: auth, drivers, matching, offers, pricing, quotes, ratings, realtime, rides, trust-score, users.

```
src/  
├─ jobs/      tarefas agendadas  
├─ lib/       config, cors, jwt, geo  
├─ modules/   um por domínio  
├─ plugins/   integrações Fastify  
└─ server.ts / app.ts  
prisma/      schema, migrations  
docs/  
├─ adr/       0001 a 0015  
├─ architecture/  
├─ operations/  
├─ planning/  
└─ quality/
```

Organização por domínio.

✦ Estrutura de pastas: App

Flutter. Pages: auth, driver, home, ride, cada uma com cubit e widgets.

```
lib/  
├─ core/           config, models, network  
├─ features/  
│   └─ pages/      uma pasta por tela  
└─ shared_widgets/ reutilizáveis
```

Organização por feature.

07

— Entrega e futuro

O que saiu do papel, e o que vem

O que foi entregue no prazo

- MVP funcional: backend e app, nos dois flavors, passageiro e motorista.
- Documentação: contrato de API, eventos WebSocket e 15 ADRs registrados.
- CI/CD: pipeline por PR e deploy automático em produção.
- Observabilidade: dashboard e alertas no Grafana.
- Testes automatizados no pipeline.

Plano de arquitetura evolutiva, próximos 6 meses



08

— Demonstração

A solução em funcionamento

Demonstração em vídeo

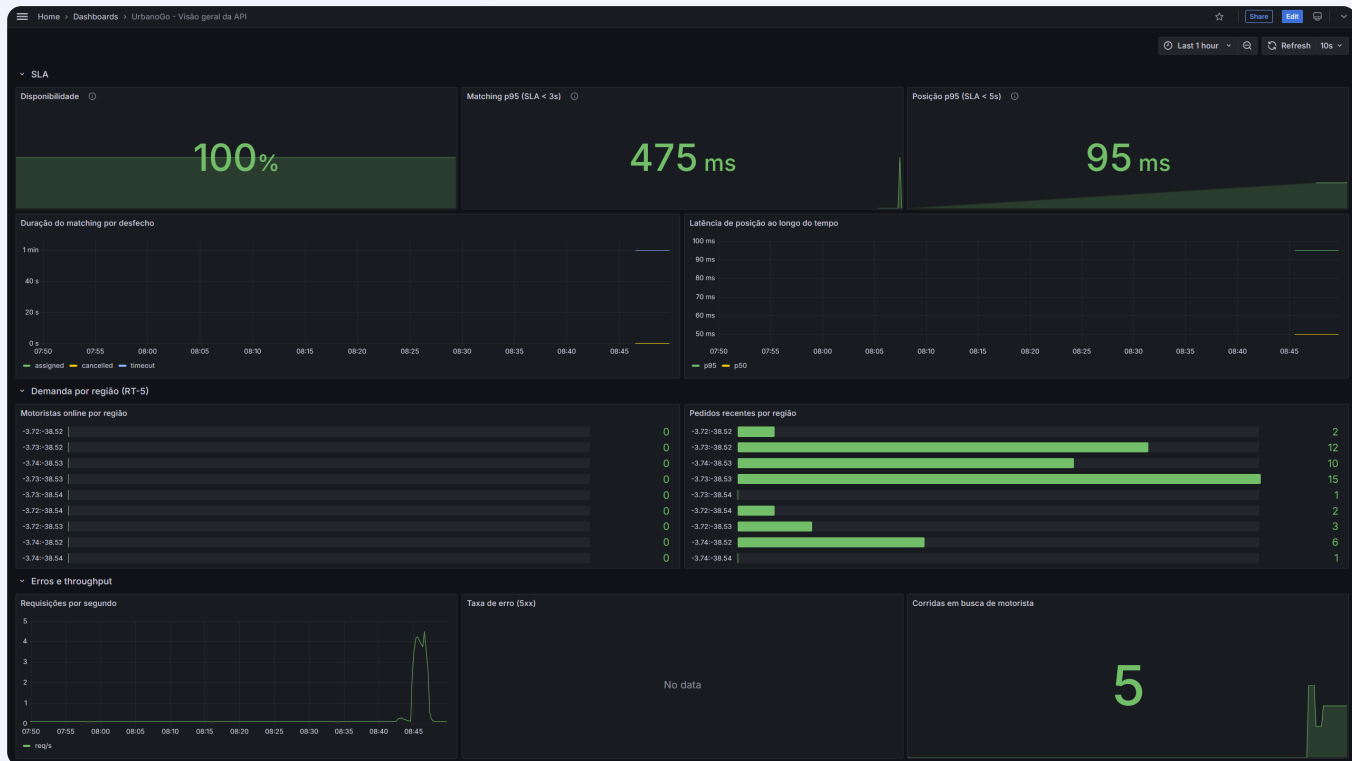
👤 Vídeo 1 · Passageiro

- Cadastro ou login real no app do passageiro
- Solicitar corrida: origem, destino e cotação de preço
- Status saindo de "procurando" para "motorista atribuído"

👤 Vídeo 2 · Motorista

- Motorista recebe a oferta e aceita
- Corrida acompanhada no mapa em tempo real até concluída
- Avaliação pós-corrida

Observabilidade: dashboard



Observabilidade: alertas

The screenshot shows the 'Alert rules' page in Grafana. The breadcrumb navigation at the top reads 'Home > Alerting > Alert rules'. The page title is 'Alert rules' with a subtitle 'Rules that determine whether an alert will fire'. A '+ New alert rule' button is in the top right. Below the title, there are filters for 'Search by data sources' (set to 'All data sources'), 'Dashboard' (set to 'Select dashboard'), 'State' (with buttons for 'Firing', 'Normal', 'Pending'), 'Rule type' (with buttons for 'Alert', 'Recording'), and 'Health' (with buttons for 'Ok', 'No Data', 'Error'). A 'Contact point' dropdown is set to 'Choose'. A search bar is present with the text 'Q Search'. To the right of the search bar, there are view options: 'View as' with 'Grouped' selected, 'List', and 'State'. Below these, it shows '3 rules' with '1 no data' and '3 normal' status indicators. A button 'Export rules' is on the right. The main section is titled 'Grafana-managed' and shows a dropdown for 'UrbanoGo > urbanogo-sla' with filters for '1 no data', '3 normal', '5m', and 'Provisioned'. A table lists three alert rules:

	State	Name	Health	Summary	Next evaluation	Actions
>	Normal	Matching p95 acima de 3s	Provisioned	ok	Latência do matching (p95, corridas atribuídas) passou do SLA de 3s	in a few seconds @ More
>	Normal	Taxa de erro acima do limite	Provisioned	nodata	Mais de 5% das respostas HTTP nos últimos 5 minutos são 5xx	in a few seconds @ More
>	Normal	API fora do ar	Provisioned	ok	O Prometheus não conseguiu raspar a API na última leitura	in a few seconds @ More

Below the table, there is a section 'Data source-managed' with the text 'No rules found.' and a '+ New recording rule' button.

Obrigado

UrbanoGo · Equipe 06